

Client Device Discovery: VSF Draft Document

Preamble

- Definitions
- Introduction
- Design Overview

Related Documents:

- [Registration and Pairing Process](#)
- [SDK API](#)
- [Host Service API](#)
- [Message Protocol](#)
- [SDK Reference Design Implementation](#)
- Testing & Validation

Definitions:

- **Cloud/Host Service (CHS):** A cloud service that Registers and Manages devices.
- **Device Operator (DO):** A person with physical or local device console access to the device.
- **Cloud/Host Operator (CO):** A person with a host service account, will claim and manage devices from the cloud.
- **SDK:** Client SDK Application the implements the client side of this TR.
- **Device:** The software/hardware onto which the SDK is installed and runs. Typically a camera, encoder, decoder.
- **Device Application:** The native device control plane process that talks to the installed SDK.
- **Certs/Certificates:** **Certs:** Access certificates. These are a collection of files created by the Cloud Host Service (CHS) to grant identity and access to the CHS. Certs are created when a device is “claimed” and passed to the device. The device uses certs to subsequently connect and communicate with the service. They can be suspended, revoked and rotated. Cert rotation refers to periodically update to improve security generally against impersonation attacks.

Introduction

AWS Elemental Media Services transport, process and deliver live and on-demand content in the cloud. Customer tell us that the hardest part of cloud workflows is often getting their live signals connected in the first place. While there are many transport protocols available for securely streaming video (SRT, RIST, TR-07, etc), customers need to manually manage device access, correctly perform stream up on both the source/destination include IP address, encryption keys and overcome networking obstacles like firewalls, network address translation and security groups. We have started an activity group with the Video Services Forum (VSF) to develop a technical recommendation and an open-source SDK application providing device discovery, authentication and connection management that works with any transport protocol.

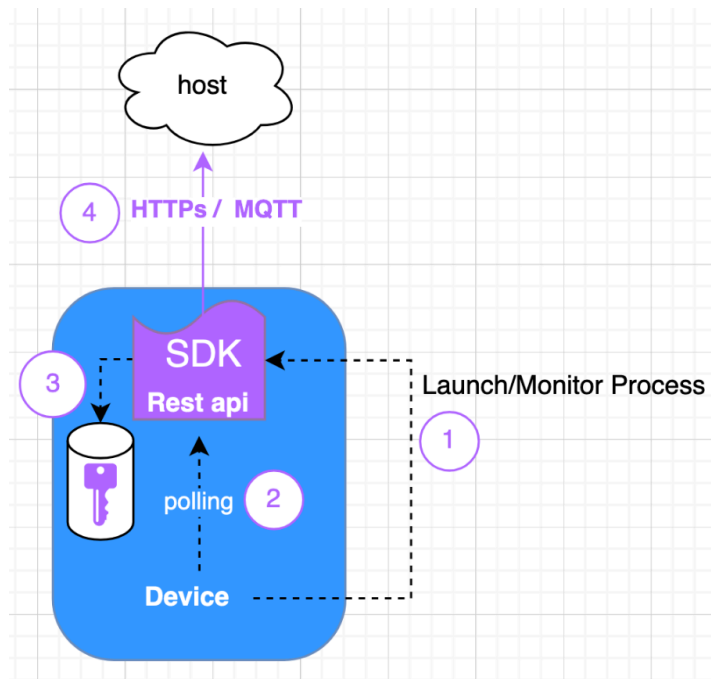
Overview of proposal

As shown in the diagram below, this Technical Recommendation (TR) describes a *system* for secure discovery and management of broadcast devices across the internet or a VPC. The system includes **(1)** an Open-Source Client-Side SDK Reference Design, **(2)** a Message and Communication Protocol and a **(3)** Supporting both a local LAN or public cloud/host service. The combined system provides a mechanism for devices to be discovered across network boundaries and communicate with a cloud/host service (CHS). The system enables the device and service to securely communicate status and configuration

Fig 1: Network Diagram: Purple items (1,2,3) are defined by the TR. Cloud/Host Service (CHS) can implement (4, 5, 6) as they see fit and are not specified in the TR.

The SDK is launched as a separate process **(1)**. Inter-process Communication (IPC) with the host application is via the SDK Rest API **(2)**. The SDK can save certs and to/from the device's filesystem **(3)**. Communication between the SDK and the Cloud Host Service is via HTTPs (pairing) and MQTT TLS/SSL **(4)**

(WIP: the SDK will also enable thumbnail subscriptions which likely will read the image from the device's filesystem and transmit to the host service. The requirement would be a `--thumbnail_path` parameter referring to a readable image file)

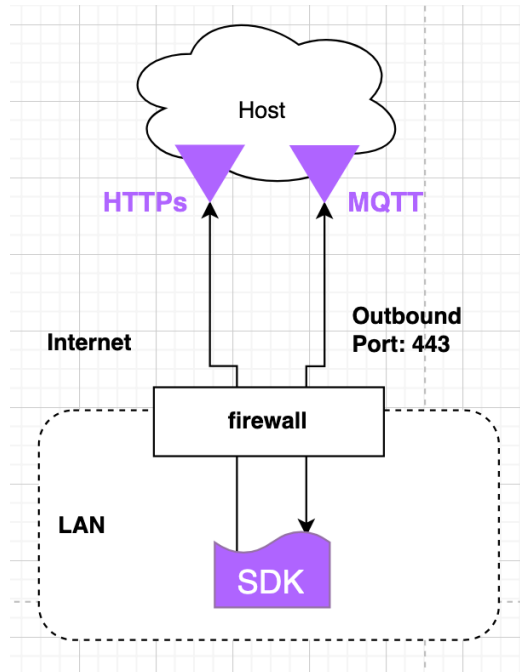


Running as a separate process with IPC via Rest reduces risk to the host application over say compiling directly as any runtime problems are isolated to the SDK and generally handled by the OS. This also enables a degree of language independence and reduces integration workload. Additionally Rest is ubiquitous, secure and convenient with built in features like request timeouts. Finally the SDK can be packaged as a Docker Image further reducing system integration effort.

Client-Host Communication Protocols

The following diagram shows how the client SDK communicates with a Cloud Service (CHS):

- **HTTPS:** Pairing and Authentication: Simple, appropriate for anonymous GET requests.
- **MQTT:** for bi-directional secure communication. This protocol is tailor-made for authenticated, secure, persistent, bi-directional communication between portable Internet of Things (IOT) Devices who's identity and access is managed by a host-service.
- **Outbound Port 443:** Provided the network enables https traffic on port 443 outbound both the HTTPS pairing and MQTT communication will work.
 - *This kind of NAT traversal via "outbound" Port 443 outbound commonly available in on-prem network settings. This is as opposed to letting traffic that originates from outside on port 443 to enter the network with is typically not allowed and in this case not needed. Importantly, port forwarding in general is not needed under this TR.*

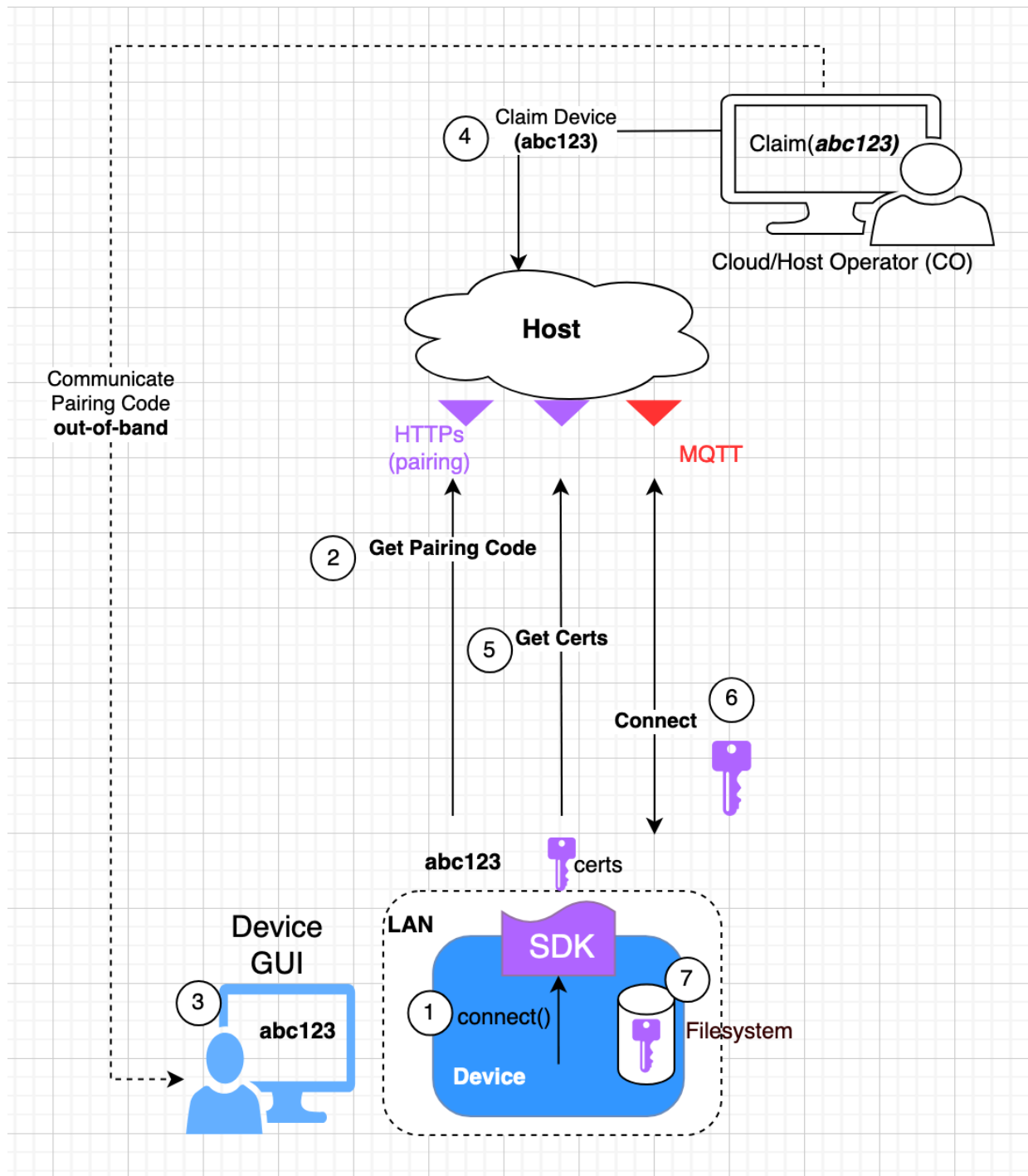


Registration & Pairing: See Discovery: Registration and Pairing Process

Pairing Overview: is the process by which a client/SDK can securely obtain an identity and certs to that it can subsequently connect to a Cloud Service (CHS) and claimed by a Cloud Operator (CO). The client gets a pairing-code from the CS from the **GetPairingCode** endpoint, then immediately polls the CS for credentials on the **Authenticate** endpoint. A CO claims a device using the pairing-code. This process is more complicated and fully described in [Discovery: Registration and Pairing Process](#) and [Discovery: Host Service API](#) for pairing API request/response details.

Scenario: Initial Connection

The pairing process is very simple for DO/CO users and generally follows the OAUTH2 design pattern. The Device Operator (DO) pairs the device by using some mechanism in the device's UI for example: a "connect my device to the cloud" button. The DO sees a new *PAIRING* state and is shown a short six character pairing code: *ABC123*. The DO passes *ABC123* the CO out-of-band. The CO Authorizes or "Claims" *ABC123* in their Cloud Service (CHS). In typically less than one second, the device shows *CONNECTED* and appears to the CO's service registry where it can be seen and managed. At this point the device is reporting it's status and ready to accept a configuration.



Host Service API: See Discovery: Host Service API

Defines how the client/service communicate and includes required endpoints, request/response definition for all operations between the client SDK and the cloud host service.

- Rest Endpoints for Pairing
- MQTT Endpoint, configuration and publish/subscribe (pub/sub) topics for status, configuration and administrative payloads.
- Request and Response parameters
- Host Configuration File: A specification provided by a host service, required.

Message Protocol: See Discovery: Message Protocol

In order to satisfy the key objectives above, this TR describes the schema methodology. This is comprised of a Protocol Schema and a Scoped Schema. These describe the structure and message content passed between the client and service via MQTT. Together the protocol and scoped schema define a set standard for a wide range of possible parameters, but enable a client to advertise support for specifically the subset of settings appropriate for the device.

The schemas are **not** open parametric definition for an arbitrary set of field-value pairs. Rather they provide all the detail necessary for control of most streaming devices.

- **Protocol schema:** Versioned, Published in the Discovery Git Repository. Defined by the TR as a standard.
- **Scoped schema:** A valid subset of the protocol schema.
- **Validations:** The SDK will validate payloads against the scoped schema. The SDK will validate that the scoped schema is a valid subset of the protocol schema
- **Extensions:** The scoping schema can define extensions for status and configuration. This enables a device to expose customized parameters. They can be used to add new settings and even hierarchal controls adjacent to parameters/hierarchies defined in the protocol.

Discovery SDK: See Discovery: SDK Reference Design

The SDK implementation is a reference design that implements the APIs and Protocols describe but this TR. A device integrator should be able to quickly install and use the reference design as production discovery client.

- The SDK includes an device application reference design that show how to start and interact with the SDK Rest API
- The SDK includes a host service capability for testing. (WIP: A running test service or AWS SAM configuration that lets anyone quickly deploy their own Discovery Service for testing)
- The SDK will include a unit tests, and integration tests for interoperability with a client application and host service
- <https://github.com/vsf-tv/gccg-cdd>

Example TRs

- TR-07: https://www.vsf.tv/download/technical_recommendations/VSF_TR-07_2022-04-20.pdf
- TR-11 (Draft): https://www.vsf.tv/download/technical_recommendations/VSF_TR-07_2022-04-20.pdf

